Day 9: Advanced GKE & Security Best Practices

---

1. Workload Identity, RBAC, and Pod Security

Workload Identity

Definition

Workload Identity allows Kubernetes workloads to securely access GCP services using service accounts without storing credentials.

How It Works

- Kubernetes Service Account (KSA)
- Linked to GCP Service Account (GSA)
- Pods inherit permissions securely

Example

- Pod accesses Cloud Storage without key files

Scenario

Application accessing GCP APIs

Problem:

- Storing service account keys in containers

Solution:

- Use Workload Identity

Result:

- Secure and keyless authentication

Case Study

FinTech Application

- Enabled Workload Identity

Result:

- Eliminated credential leakage risks

RBAC (Role-Based Access Control)

Definition

RBAC controls access to Kubernetes resources based on roles and permissions.

Key Components

- Role / ClusterRole
- RoleBinding / ClusterRoleBinding

Example

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

Scenario

Developer accidentally deletes resources

Problem:

- Excess permissions

Solution:

- Restrict access using RBAC

Result:

- Controlled access

Case Study

Enterprise Cluster

- Implemented RBAC roles

Result:

- Secure multi-user access

Pod Security (Policies / Standards)

Definition

Controls what Pods are allowed to do (privileged access, root user, etc.).

Best Practices

- Run containers as non-root
- Avoid privileged containers
- Restrict host access

Scenario

Malicious container behavior

Solution:

- Enforce pod security policies

Result:

- Improved cluster security

---

2. Private Clusters and Authorized Networks

Private Cluster

Definition

Cluster where nodes do not have public IP addresses.

Benefits

- Increased security
- Reduced exposure to internet

Authorized Networks

Definition

Restrict access to Kubernetes API server based on IP ranges.

Scenario

Public access to cluster

Problem:

- Security risk

Solution:

- Use private cluster + authorized networks

Result:

- Restricted access

Case Study

Banking System

- Uses private GKE cluster

Result:

- Secure infrastructure

---

3. Network Policies and Firewall Rules

Network Policies

Definition

Control communication between Pods inside the cluster.

Example

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      role: frontend
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: backend
```

Firewall Rules

Definition

Control network traffic at infrastructure level.

Scenario

Unrestricted communication

Problem:

- Any Pod can talk to any Pod

Solution:

- Apply network policies

Result:

- Secure communication

Case Study

Microservices Architecture

- Restricted communication between services

Result:

- Reduced attack surface

---

4. GKE Hardening and CIS Benchmarks

Definition

Hardening = Securing Kubernetes cluster using best practices and standards.

CIS Benchmarks = Industry guidelines for securing systems.

Key Practices

- Enable RBAC
- Use private clusters
- Restrict network access
- Regular vulnerability scanning
- Use least privilege principle

Scenario

Security audit failure

Problem:

- Weak configurations

Solution:

- Apply CIS benchmarks

Result:

- Compliance achieved

Case Study

Healthcare Organization

- Implemented GKE hardening

Result:

- Passed security audits
- Improved compliance

---

5. Hands-On: Workload Identity and RBAC

Step 1: Enable Workload Identity

```
gcloud container clusters update my-cluster \
  --workload-pool=<project-id>.svc.id.goog
```

Step 2: Create GCP Service Account

```
gcloud iam service-accounts create my-gsa
```

Step 3: Bind IAM Role

```
gcloud projects add-iam-policy-binding <project-id> \
  --member="serviceAccount:my-gsa@<project-id>.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

Step 4: Create Kubernetes Service Account

```
kubectl create serviceaccount my-ksa
```

Step 5: Link KSA to GSA

```
kubectl annotate serviceaccount my-ksa \
  iam.gke.io/gcp-service-account=my-gsa@<project-id>.iam.gserviceaccount.com
```

Step 6: Deploy Pod Using KSA

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: my-pod
spec:
  serviceAccountName: my-ksa
  containers:
  - name: app
    image: <image-url>
```

Step 7: Implement RBAC

Create Role:

```
kubectl apply -f role.yaml
```

Bind Role:

```
kubectl apply -f rolebinding.yaml
```

End-to-End Case Study (Day 9 Summary)

Scenario: Secure GKE Deployment

Problem:

- Unrestricted access
- Credential exposure
- Weak security controls

Solution

1. Enable Workload Identity
2. Implement RBAC
3. Use private clusters
4. Apply network policies
5. Follow CIS benchmarks

Result

- Secure authentication
- Controlled access
- Reduced attack surface
- Compliance with security standards